

SOFTWARE MAINTENANCE

Classical Tech Debt & AI-Generated Rot

"Software Engineering in the Age of AI"

CLASSICAL MAINTENANCE



The 80/20 Reality

Maintenance is the longest phase of the software lifecycle.

- **Corrective:** Bug fixes.
- **Adaptive:** New environments/platforms.
- **Perfective:** Performance & features.
- **Preventive:** Refactoring for the future.



Lehman's Law

As a system evolves, its complexity increases unless work is done to maintain or reduce it.

CLASSICAL: TECHNICAL DEBT

The "Interest" of Speed

Technical debt is the rework cost caused by choosing an easy solution now instead of a better approach that takes longer.

Signs of Debt:

- **Rigidity:** Hard to change.
- **Fragility:** Easy to break.
- **Immobility:** Hard to reuse.

Refactor

Paying down the debt.

Restructuring code without changing external behavior.

CLASSICAL: PRACTICAL REFACTORING

```
# BEFORE: Messy & Fragile def calc(a, b, t): if t == 's':  
return a + b elif t == 'm': return a * b # Cryptic names, hard  
to extend
```

```
# AFTER: Clean & Extensible def calculate_total(price,  
tax, mode): ops = { 'sum': lambda x, y: x + y, 'multiply':  
lambda x, y: x * y } return ops[mode](price, tax)
```

Eliminating "Code Smells" [Refactoring Guru]

MAINTENANCE IN THE AI ERA

Managing "Rot" in Generated Code

THE NEW CHALLENGE: AI "ROT"



What is AI Rot?

Accumulation of generated code that is functional but not understood, verified, or integrated into the project's logic.



Velocity vs. Quality

AI generates code faster than humans can review it, creating **Invisible Tech Debt**.

"Using AI is required. Trusting AI is failing."

AI MAINTENANCE STRATEGY

1. Review over Coding

Focus shifts from writing lines to auditing them. Treat AI as a "junior" whose work must be verified.

2. Clean as you Go

Refactor messy AI-generated scripts into clean Classes immediately. Don't let raw scripts survive the commit.

3. Automated Verification

Use "Evals" to check for logic drift. If the AI changes a function, the regression suite must catch it.

COMPARING MAINTENANCE MODELS

Feature	Classical Maintenance	AI-Native Maintenance
Debt Source	Human shortcuts & deadlines	Volume of generated "Black Box" code
Key Activity	Manual Refactoring	Verification & Intent Auditing
Main Tool	Unit Testing / IDE Refactors	AI Evals / Property-based Testing
Risk	Complexity Stagnation	Systemic "Rot" and Hallucinations

PRACTICAL GOAL: THE CLEANUP

The Refactoring Task

Take a functional but messy AI-generated script and refactor it into clean, documented, and tested Classes.

Definition of Done

- Zero "Code Smells".
- Verified intent (Does it do what was asked?).
- Full regression suite passes.

SUMMARY

Maintenance is the **Primary** stage of engineering.

Whether it's human tech debt or AI rot, the discipline of verification is non-negotiable.
